YNOV CAMPUS / JOUR 3

Jour 3 Écrire des analyses avancées

CTE, récursivité, fenêtres et rapports multidimensionnels. Apprenez avec des exemples commentés, puis exercez-vous dans TP03.

PostgreSQL 18 | 2e année de bachelor | Édition pédagogique progressive

Partir d’une question nécessitant deux étapes

Quelles commandes payées de CampusMarket ont rapporté plus de 60 ? Calculez d’abord un total par commande, puis filtrez ces totaux. Une table dérivée est un SELECT placé dans FROM pour que son résultat serve d’entrée à un autre SELECT. Elle fournit à la requête externe des colonnes ordinaires à exploiter, dont un total absent de la table des commandes stockée.

Une table dérivée est le résultat d’une sous-requête utilisé comme table dans FROM.

Sens des mots-clés

table dérivée — Résultat d’une requête utilisé comme entrée dans FROM.

requête externe — Le SELECT qui lit le résultat d’un autre SELECT.

granularité — Ce que représente une ligne d’un résultat.

Calculer les totaux des commandes avant de les filtrer

Lisez d’abord le SELECT entre parenthèses. Il joint chaque commande payée à ses lignes et additionne la quantité multipliée par le prix enregistré sur cette ligne. L’alias order_totals nomme ce résultat de requête. Le WHERE externe peut désormais utiliser total comme colonne d’entrée. Les paiements sont volontairement absents : joindre leurs lignes répéterait certaines lignes de commande et gonflerait le chiffre d’affaires.

SQL / lire l’instruction une clause à la fois

```sql
SELECT order_id,total
FROM (
 SELECT o.order_id,
  SUM(i.quantity*i.unit_price) AS total
 FROM campus.orders AS o
 JOIN campus.order_items AS i USING (order_id)
 WHERE o.status='paid'
 GROUP BY o.order_id
) AS order_totals
WHERE total>60
ORDER BY order_id;
```

Calculer les totaux des commandes avant de les filtrer / raisonnement pas à pas

Suivez ces étapes en parallèle de l’exemple SQL précédent. Prévoyez ce que chaque étape reçoit et produit avant de l’exécuter. L’essentiel n’est pas seulement de savoir si l’instruction s’exécute, mais pourquoi ces opérations répondent à la question posée avec les données fournies.

Suivez le raisonnement

1. La requête interne produit un total pour chaque commande payée.
2. La requête externe ne conserve que les totaux supérieurs à 60.
3. Le tri final rend la sortie reproductible.

Lire le résultat comme quatre commandes payées

Quatre commandes passent le filtre. La commande 104 contribue pour 75 ; les commandes 101, 102 et 107 contribuent chacune pour 70. Ce résultat contient une ligne par commande retenue, pas une ligne par produit acheté. Les autres commandes payées existent toujours dans la base de données ; elles ne satisfont simplement pas le seuil de cette question. Modifier la condition externe ne change pas le calcul des totaux internes.

Résultat / données d’exemple

order_id	total
101	70
102	70
104	75
107	70

Calculez à la granularité requise, puis filtrez les lignes calculées.

WITH donne un nom lisible à une étape de requête

Une expression de table commune, abrégée CTE, est un résultat de requête nommé disponible dans une seule instruction SQL. WITH introduit ce nom et AS encadre le SELECT qui le définit. Une liste facultative de colonnes après le nom de la CTE nomme ses colonnes de sortie dans l’ordre. Le SELECT final lit la CTE comme une table. Cette version répond à la même question que la table dérivée, mais place le calcul avant la requête qui l’utilise.

SQL / lire l’instruction une clause à la fois

```sql
WITH order_totals AS (
 SELECT o.order_id,
  SUM(i.quantity*i.unit_price) AS total
 FROM campus.orders AS o
 JOIN campus.order_items AS i USING (order_id)
 WHERE o.status='paid'
 GROUP BY o.order_id
)
SELECT order_id,total FROM order_totals
WHERE total>60
ORDER BY order_id;
```

WITH donne un nom lisible à une étape de requête / guide des mots-clés

Utilisez ce vocabulaire pour lire l’exemple précédent. Chaque définition explique le rôle exact d’un mot ou symbole dans l’instruction. Relisez le SQL depuis sa table d’entrée jusqu’à ses conditions et sa sortie, en utilisant ces définitions pour expliquer ce qui est demandé au serveur.

Sens des mots-clés

WITH — Introduit des étapes de requête nommées pour cette instruction.

AS (...) — Définit la requête fournissant les lignes d’une étape.

CTE — Expression de table commune, utilisable dans une seule instruction.

name(col1,col2) — Nomme la CTE et ses colonnes de sortie dans leur ordre de position.

Une CTE prend fin avec son instruction

La portée indique où un nom est disponible. Une CTE n’existe que dans l’instruction introduite par sa clause WITH ; le point-virgule termine cette instruction. Ce n’est pas une table stockée qu’une autre instruction pourra interroger plus tard. Plusieurs CTE peuvent suivre un même WITH, séparées par des virgules, et une étape ultérieure peut lire une étape précédente. Nommez les étapes d’après les lignes qu’elles contiennent.

Sens des mots-clés

portée — Partie d’une instruction où un nom peut être utilisé.

instruction — Commande SQL complète, généralement terminée par un point-virgule.

Suivez le raisonnement

1. Donnez à la première étape un nom décrivant la granularité de ses lignes.
2. Lisez cette étape dans le SELECT final ou dans une CTE ultérieure.
3. Répétez la définition WITH au début d’une instruction distincte.

À vous de jouer : présenter les petites commandes payées

Utilisez la CTE order_totals de l’exemple précédent. Modifiez uniquement le SELECT final pour renvoyer les commandes payées dont le total est inférieur à 60. Affichez order_id et total, placez les plus grands totaux d’abord et départagez les totaux égaux par order_id croissant. Avant d’exécuter la requête, prévoyez le nombre de lignes et expliquez pourquoi les commandes pending et cancelled ne peuvent pas figurer dans le résultat.

EXTRAIT / assemblez les sections expliquées avant l’exécution

```
-- Fragment : remplacez seulement le SELECT final.
SELECT order_id,total
FROM order_totals
WHERE /* write the total condition */
ORDER BY /* total, then order_id */;
```

Instructions d’exécution

example-fragment : remplacez le SELECT final dans d3_cte_orders ; les commentaires indiquent les blancs à compléter.

Un nouveau rapport peut réutiliser le même calcul défini.

Corrigé expliqué : conserver le même calcul

La condition est total < 60 et le tri est total DESC, order_id. Les commandes 103, 109 et 110 totalisent chacune 50 ; la commande 108 totalise 40. La CTE élimine toujours les commandes pending et cancelled avant la somme. Cet exemple montre l’utilité d’une étape nommée : une définition fiable du chiffre d’affaires des commandes payées peut servir à plusieurs questions de rapport bien distinctes.

SQL / lire l’instruction une clause à la fois

```
WITH order_totals AS (
 SELECT o.order_id,
  SUM(i.quantity*i.unit_price) AS total
 FROM campus.orders AS o
 JOIN campus.order_items AS i USING (order_id)
 WHERE o.status='paid'
 GROUP BY o.order_id
)
SELECT order_id,total FROM order_totals
WHERE total<60
ORDER BY total DESC,order_id;
```

Corrigé expliqué : conserver le même calcul / résultat commenté

La condition est total < 60 et le tri est total DESC, order_id. Les commandes 103, 109 et 110 totalisent chacune 50 ; la commande 108 totalise 40. La CTE élimine toujours les commandes pending et cancelled avant la somme. Cet exemple montre l’utilité d’une étape nommée : une définition fiable du chiffre d’affaires des commandes payées peut servir à plusieurs questions de rapport bien distinctes.

Résultat / données d’exemple

order_id	total
103	50
109	50
110	50
108	40

Une fenêtre calcule sur plusieurs lignes sans les supprimer

Une fonction de fenêtrage calcule une valeur à partir des lignes liées à la ligne courante tout en conservant les lignes individuelles dans la sortie. Supposons qu’un rapport doive afficher les ventes de chaque personne avec le total de son équipe à côté. GROUP BY remplacerait les personnes par des lignes de synthèse par équipe. Un SUM fenêtré peut répéter le total d’équipe sur la ligne de chaque personne, préservant le détail nécessaire à la comparaison.

Une fonction de fenêtrage ajoute des calculs sur des lignes liées tout en conservant le détail des lignes d’entrée.

Sens des mots-clés

fonction de fenêtrage — Calcul sur des lignes liées qui conserve chaque ligne de détail.

ligne courante — Ligne de sortie pour laquelle le calcul est évalué.

partition — Groupe de lignes utilisé indépendamment par un calcul de fenêtrage.

OVER et PARTITION BY définissent le groupe de comparaison

OVER transforme ce SUM en calcul de fenêtrage. PARTITION BY team répartit les six lignes d’entrée en groupes de calcul séparés : une équipe ne contribue jamais au total d’une autre. Contrairement à GROUP BY, il ne regroupe pas les personnes de chaque équipe en une ligne. L’en-tête sales(team,person,amount) nomme dans l’ordre les trois colonnes VALUES de la CTE. Le ORDER BY final ne fait que trier les lignes affichées.

SQL / lire l’instruction une clause à la fois

```sql
WITH sales(team,person,amount) AS (
 VALUES ('A','Ada',40),('A','Ben',30),
        ('A','Cam',30),('B','Dee',50),
        ('B','Eli',50),('B','Fay',20)
)
SELECT team,person,amount,
 SUM(amount) OVER (PARTITION BY team) AS team_total
FROM sales
ORDER BY team,person;
```

Sens des mots-clés

OVER (...) — Spécifie les lignes disponibles pour une fonction de fenêtrage.

PARTITION BY — Lance un calcul distinct pour chaque groupe distinct.

OVER () — Utilise toutes les lignes d’entrée comme une seule partition de fenêtre.

Observer le cadre s’étendre tout en conservant les lignes des mois

Un total cumulé est calculé pour chaque ligne courante à partir de son cadre de fenêtre. Ici, le cadre commence en janvier et s’arrête au mois courant. Février inclut janvier et février ; mars inclut les trois mois. Le résultat contient toujours trois lignes de mois, car un calcul de fenêtrage préserve le sens des lignes d’entrée.

Mois	Chiffre d’affaires	Lignes incluses dans le cadre de mars	Total cumulé
Janv.	190	Janv.	190
Févr.	145	Janv. + Févr.	335
Mars	140	Janv. + Févr. + Mars	475

La fenêtre ajoute un total à chaque ligne de mois existante. Elle ne réduit pas les trois mois à une seule ligne.

Le total d’équipe se répète ; les personnes restent

L’équipe A totalise 100 et l’équipe B, 120. Chaque personne reste dans le résultat et reçoit le total de sa propre équipe. Additionner verticalement la colonne team_total affichée compterait chaque équipe trois fois : c’est une colonne de comparaison, pas un nouveau fait de vente. Un rapport ultérieur doit toujours décider si ses lignes représentent des personnes, des équipes ou des transactions.

Résultat / données d’exemple

team	person	amount	team_total
A	Ada	40	100
A	Ben	30	100
A	Cam	30	100
B	Dee	50	120
B	Eli	50	120
B	Fay	20	120

Les calculs de fenêtrage conservent les lignes ; les agrégats groupés les remplacent.

Choisir le sens d’un ex æquo avant de classer

Un classement exige un ordre et une règle pour les montants égaux. RANK attribue la même position aux lignes ex æquo et laisse un saut ensuite. DENSE_RANK partage aussi les positions, sans saut. ROW_NUMBER donne un numéro différent à chaque ligne. Le ORDER BY dans OVER contrôle ces calculs ; le ORDER BY externe contrôle l’affichage. Dans notre exemple, le nom d’une personne ne départage les ex æquo que pour ROW_NUMBER.

Sens des mots-clés

RANK() — Positions partagées avec des sauts après les ex æquo.

DENSE_RANK() — Positions partagées sans saut après les ex æquo.

ROW_NUMBER() — Un numéro séquentiel différent pour chaque ligne.

ORDER BY de fenêtre — Ordre des lignes utilisé par un calcul de fenêtrage.

ORDER BY externe — Ordre du résultat final affiché.

WINDOW — Nomme une spécification OVER pour que plusieurs fonctions la réutilisent.

Comparer trois règles de classement sur six personnes

La fenêtre nommée w trie les personnes par amount décroissant et est réutilisée par RANK et DENSE_RANK. ROW_NUMBER utilise son propre tri avec person en plus : les montants égaux reçoivent ainsi des numéros distincts reproductibles. Le tri externe utilise le même critère pour départager l’affichage. Ces choix distinguent les positions de vente partagées du classement des numéros de ligne uniques nécessaires pour sélectionner un nombre précis de personnes.

SQL / lire l’instruction une clause à la fois

```sql
WITH s(person,amount) AS (
 VALUES ('Ada',40),('Ben',30),('Cam',30),
        ('Dee',20),('Eli',20),('Fay',10)
)
SELECT person,amount,RANK() OVER w AS r,
 DENSE_RANK() OVER w AS dr,
 ROW_NUMBER() OVER (
  ORDER BY amount DESC,person) AS rn
FROM s
WINDOW w AS (ORDER BY amount DESC)
ORDER BY amount DESC,person;
```

Comparer trois règles de classement sur six personnes / guide des mots-clés

Utilisez ce vocabulaire pour lire l’exemple précédent. Chaque définition explique le rôle exact d’un mot ou symbole dans l’instruction. Relisez le SQL depuis sa table d’entrée jusqu’à ses conditions et sa sortie, en utilisant ces définitions pour expliquer ce qui est demandé au serveur.

Sens des mots-clés

WINDOW w AS (...) — Nomme une spécification de fenêtre w pour que plusieurs calculs OVER w la réutilisent.

OVER w — Utilise la spécification de fenêtre nommée w.

Les ex æquo donnent des positions différentes après eux

Ben et Cam reçoivent tous deux le rang 2. Dee suit au rang 4, car trois personnes la précèdent, mais au rang dense 3, car seuls deux montants distincts sont supérieurs. ROW_NUMBER attribue la séquence de 1 à 6 en utilisant les noms pour départager les montants égaux. Demander les deux premiers rangs peut donc renvoyer plus de deux personnes ; demander deux numéros de ligne ne le peut pas.

Résultat / données d’exemple

person	amount	r	dr	rn
Ada	40	1	1	1
Ben	30	2	2	2
Cam	30	2	2	3
Dee	20	4	3	4
Eli	20	4	3	5
Fay	10	6	4	6

La règle des ex æquo fait partie de la question métier.

À vous de jouer : conserver tous les membres des deux premiers rangs

L’équipe commerciale veut tous les membres des deux premiers rangs de ventes, ex æquo compris. Utilisez les données des six personnes de l’exemple de classement, calculez RANK dans une CTE appelée ranked et filtrez son résultat avec r <= 2. Expliquez pourquoi le filtre appartient au SELECT qui lit ranked. Prévoyez si Cam reste dans la sortie avant d’exécuter l’instruction.

Suivez le raisonnement

1. Copiez l’entrée VALUES de six lignes dans une CTE nommée s.
2. Ajoutez une CTE ranked qui sélectionne person, amount et RANK.
3. Filtrez ranked avec r <= 2 et triez par r,person.

Calculez un résultat de fenêtre à une étape, puis filtrez-le à la suivante.

Corrigé expliqué : classer, puis filtrer

Ada, Ben et Cam sont retenus, car leurs rangs de ventes partagés sont 1, 2 et 2. La deuxième CTE fait de r une colonne d’entrée pour le WHERE final. Avec la règle ROW_NUMBER <= 2, Cam serait exclu selon notre critère de départage par nom. Une requête par étapes rend donc visibles le calcul et le sens métier de son filtre final.

SQL / lire l’instruction une clause à la fois

```sql
WITH s(person,amount) AS (
 VALUES ('Ada',40),('Ben',30),('Cam',30),
        ('Dee',20),('Eli',20),('Fay',10)
), ranked AS (
 SELECT person,amount,
  RANK() OVER (ORDER BY amount DESC) AS r
 FROM s
)
SELECT person,amount,r FROM ranked
WHERE r<=2
ORDER BY r,person;
```

Corrigé expliqué : classer, puis filtrer / résultat commenté

Ada, Ben et Cam sont retenus, car leurs rangs de ventes partagés sont 1, 2 et 2. La deuxième CTE fait de r une colonne d’entrée pour le WHERE final. Avec la règle ROW_NUMBER <= 2, Cam serait exclu selon notre critère de départage par nom. Une requête par étapes rend donc visibles le calcul et le sens métier de son filtre final.

Résultat / données d’exemple

person	amount	r
Ada	40	1
Ben	30	2
Cam	30	2

LAG et LEAD lisent les lignes voisines

Comment les ventes ont-elles évolué d’un mois à l’autre ? LAG(amount) lit le montant de la ligne précédente dans l’ordre ; LEAD(amount) lit celui de la suivante. Nos six lignes représentent les mois 1 à 6, y compris les mois sans vente. Soustraire LAG du montant courant donne la variation. Un voisin absent produit NULL par défaut, car aucune valeur précédente ou suivante n’existe.

SQL / lire l’instruction une clause à la fois

```sql
WITH s(m,amount) AS (
 VALUES (1,10),(2,20),(3,0),
        (4,40),(5,50),(6,0)
)
SELECT m,amount,LAG(amount) OVER w AS before,
 LEAD(amount) OVER w AS after,
 amount-LAG(amount) OVER w AS change
FROM s
WINDOW w AS (ORDER BY m)
ORDER BY m;
```

Sens des mots-clés

LAG(value) — Lit la valeur de la ligne précédente dans l’ordre de la fenêtre.

LEAD(value) — Lit la valeur de la ligne suivante dans l’ordre de la fenêtre.

NULL par défaut — Résultat lorsque la ligne voisine demandée est absente.

Une variation négative signifie une baisse par rapport à la ligne précédente

Le mois 3 passe de 20 à 0, soit une variation de -20. Le mois 4 passe de 0 à 40, soit +40. Le premier mois n’a pas de ligne précédente et le sixième n’a pas de ligne suivante : les cellules correspondantes valent donc NULL. LEAD et LAG ne créent pas les mois manquants et ne déduisent pas les valeurs absentes ; ils examinent les lignes déjà fournies au calcul.

Résultat / données d’exemple

m	amount	before	after	change
1	10	NULL	20	NULL
2	20	10	0	10
3	0	20	40	-20
4	40	0	50	40
5	50	40	0	10
6	0	50	NULL	-50

Un cadre indique quelles lignes ordonnées entrent dans ce total

Un total cumulé ajoute chaque montant à tout ce qui a déjà été rencontré. Un cadre de fenêtre est la partie d’une partition ordonnée utilisée pour le calcul courant. ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW commence à la première ligne et s’arrête à cette ligne. Écrire explicitement le cadre rend visible le cumul voulu ligne par ligne, sans dépendre du traitement par défaut des valeurs de tri ex æquo dans PostgreSQL.

SQL / lire l’instruction une clause à la fois

```sql
WITH s(m,amount) AS (
 VALUES (1,10),(2,20),(3,0),
        (4,40),(5,50),(6,0)
)
SELECT m,amount,SUM(amount) OVER (
 ORDER BY m ROWS BETWEEN UNBOUNDED PRECEDING
 AND CURRENT ROW) AS running_total
FROM s
ORDER BY m;
```

Sens des mots-clés

cadre de fenêtre — Sous-ensemble des lignes ordonnées utilisé pour le calcul de cette ligne.

ROWS — Définit les limites du cadre selon les positions des lignes.

UNBOUNDED PRECEDING — Commence à la première ligne de la partition.

CURRENT ROW — Se termine à la ligne dont le calcul est en cours.

Des ventes nulles laissent le total cumulé inchangé

Le total cumulé atteint 30 après le mois 2 et reste à 30 au mois 3, qui ajoute zéro. Il atteint 120 au mois 5 et reste à 120 au mois 6. Chaque résultat utilise un préfixe plus long des mêmes six lignes. Contrairement au SUM de l’équipe entière vu précédemment, ce cadre ordonné donne volontairement des totaux différents selon la position dans la partition.

Résultat / données d’exemple

m	amount	running_total
1	10	10
2	20	30
3	0	30
4	40	70
5	50	120
6	0	120

Précisez le cadre lorsque le calcul dépend des limites des lignes ordonnées.

LAST_VALUE signifie la dernière valeur du cadre

LAST_VALUE(amount) renvoie le montant de la dernière ligne du cadre courant, qui n’est pas automatiquement la dernière ligne de tout le rapport. Avec ORDER BY sans cadre explicite, il renvoie couramment le montant de la ligne courante. Pour comparer chaque mois au dernier mois, étendez le cadre jusqu’à UNBOUNDED FOLLOWING, soit la dernière ligne de la partition. Le tri détermine toujours quel mois est le dernier.

SQL / lire l’instruction une clause à la fois

```sql
WITH s(m,amount) AS (
 VALUES (1,10),(2,20),(3,0),
        (4,40),(5,50),(6,0)
)
SELECT m,amount,
 LAST_VALUE(amount) OVER (ORDER BY m) AS default_last,
 LAST_VALUE(amount) OVER (
  ORDER BY m ROWS BETWEEN UNBOUNDED PRECEDING
  AND UNBOUNDED FOLLOWING) AS final_amount
FROM s ORDER BY m;
```

Sens des mots-clés

LAST_VALUE(value) — Lit value sur la dernière ligne du cadre courant.

UNBOUNDED FOLLOWING — Étend le cadre jusqu’à la dernière ligne de la partition.

Le dernier montant est zéro, pas le maximum

Le cadre explicite couvrant toute la partition renvoie 0 pour chaque mois, car le mois 6 est la dernière ligne et son montant est zéro. Le cadre par défaut renvoie chaque montant courant dans cet exemple. Le montant maximal, 50, ne concerne pas LAST_VALUE : le plus grand et le dernier répondent à des questions différentes. Une fonction de fenêtrage correcte peut répondre à la mauvaise question métier si son tri ou son cadre est inadapté.

Résultat / données d’exemple

m	amount	default_last	final_amount
1	10	10	0
2	20	20	0
3	0	0	0
4	40	40	0
5	50	50	0
6	0	0	0

Générer des dates même sans ligne de vente

Un rapport quotidien exige une ligne par jour calendaire, y compris les jours sans activité. generate_series produit une séquence de lignes du début à la fin incluse. Ici, il produit les entiers de 0 à 5. PostgreSQL ajoute chaque entier au littéral DATE, avançant d’autant de jours. Ce calendrier de six jours peut former le côté gauche d’une jointure pour garder visibles les dates sans vente.

SQL / lire l’instruction une clause à la fois

```sql
SELECT DATE '2026-01-01'+g.n AS d
FROM generate_series(0,5) AS g(n)
ORDER BY d;
```

Sens des mots-clés

generate_series(start,stop) — Renvoie des lignes d’entiers, bornes incluses, par pas de un.

g(n) — Nomme la table générée g et sa colonne n.

littéral DATE — Valeur entre guillemets explicitement typée comme date calendaire.

date + integer — Avance une date de ce nombre de jours.

Le calendrier compte six lignes avant toute jointure avec les ventes

La séquence contient les six dates dans l’ordre. Ces lignes décrivent la période du rapport, pas des ventes. Leur intérêt est la couverture : un LEFT JOIN ultérieur peut rattacher les ventes présentes tout en conservant les dates sans vente correspondante. Le choix des limites du calendrier relève des exigences du rapport ; utilisez donc des dates de début et de fin explicites au lieu de les déduire silencieusement des seules transactions existantes.

Résultat / données d’exemple

d
2026-01-01
2026-01-02
2026-01-03
2026-01-04
2026-01-05
2026-01-06

Rattacher les observations au calendrier complet

Nos observations simplifiées contiennent quatre totaux quotidiens : 10, 20, 40 et 50. Le LEFT JOIN conserve les six dates générées. COALESCE remplace un montant sans correspondance de NULL par zéro, car cet exercice définit une observation absente comme un jour sans vente confirmé. Ce remplacement est une décision métier ; dans un autre jeu de données, une mesure manquante peut signifier « inconnu » et ne doit pas automatiquement devenir zéro.

SQL / lire l’instruction une clause à la fois

```sql
SELECT DATE '2026-01-01'+g.n AS d,
 COALESCE(f.amount,0) AS amount
FROM generate_series(0,5) AS g(n)
LEFT JOIN (VALUES
 (DATE '2026-01-01',10),
 (DATE '2026-01-02',20),
 (DATE '2026-01-04',40),
 (DATE '2026-01-05',50)
) AS f(d,amount)
 ON f.d=DATE '2026-01-01'+g.n
ORDER BY d;
```

Une moyenne mobile exige un axe temporel complet

La série quotidienne complétée comprend maintenant zéro les 3 et 6 janvier. Une moyenne mobile est une moyenne sur un cadre qui avance avec la ligne courante. Avec une ligne par jour, la ligne courante et les deux précédentes représentent jusqu’à trois jours calendaires. Sans les dates ajoutées, le même cadre ROWS représenterait trois observations réparties sur une période plus longue et irrégulière.

Une moyenne mobile calcule la moyenne d’un cadre borné qui avance pour chaque ligne ordonnée.

Une moyenne mobile exige un axe temporel complet / résultat commenté

La série quotidienne complétée comprend maintenant zéro les 3 et 6 janvier. Une moyenne mobile est une moyenne sur un cadre qui avance avec la ligne courante. Avec une ligne par jour, la ligne courante et les deux précédentes représentent jusqu’à trois jours calendaires. Sans les dates ajoutées, le même cadre ROWS représenterait trois observations réparties sur une période plus longue et irrégulière.

Résultat / données d’exemple

d	amount
2026-01-01	10
2026-01-02	20
2026-01-03	0
2026-01-04	40
2026-01-05	50
2026-01-06	0

Calculer la moyenne du jour courant et des deux jours précédents

Pour rendre le cadre visible, cette requête écrit directement les six lignes quotidiennes complétées sous forme de VALUES ; il s’agit exactement du résultat de la jointure précédente, exprimé en numéros de jour. ROWS BETWEEN 2 PRECEDING AND CURRENT ROW sélectionne au plus trois lignes. AVG calcule leur moyenne et ROUND affiche deux décimales. Le premier jour n’a qu’une valeur disponible et le deuxième en a deux : leurs moyennes utilisent donc des cadres plus courts.

SQL / lire l’instruction une clause à la fois

```
WITH daily(d,amount) AS (
 VALUES (1,10),(2,20),(3,0),
        (4,40),(5,50),(6,0)
)
SELECT d,amount,ROUND(AVG(amount) OVER (
 ORDER BY d ROWS BETWEEN 2 PRECEDING
 AND CURRENT ROW),2) AS moving_avg
FROM daily
ORDER BY d;
```

Sens des mots-clés

2 PRECEDING — Commence le cadre deux positions de ligne avant cette ligne.

AVG — Additionne les valeurs non NULL et divise par leur nombre.

ROUND(value,2) — Affiche le résultat numérique avec deux décimales.

Lire chaque moyenne avec son dénominateur réel

Le jour 1 ne calcule la moyenne que de 10, soit 10.00 ; le jour 2 calcule celle de 10 et 20, soit 15.00. À partir du jour 3, chaque cadre contient trois jours. Le jour 6 calcule la moyenne de 40, 50 et 0, soit 30.00, bien que ses propres ventes soient nulles. La moyenne décrit l’activité récente dans son cadre ; amount décrit celle d’un seul jour.

Résultat / données d’exemple

d	amount	moving_avg
1	10	10.00
2	20	15.00
3	0	10.00
4	40	20.00
5	50	30.00
6	0	30.00

À vous de jouer : utiliser une moyenne mobile sur deux jours

Modifiez la requête de moyenne mobile précédente pour utiliser le jour courant et un seul jour précédent. Conservez les six lignes quotidiennes complètes, leur tri et l’affichage à deux décimales. Prévoyez les moyennes des jours 1, 3 et 6 avant d’exécuter la requête. Expliquez pourquoi il suffit de modifier la limite du cadre et pourquoi retirer les jours sans vente changerait le sens du calcul.

Suivez le raisonnement

1. Remplacez 2 PRECEDING par la limite correspondant à une ligne antérieure.
2. Calculez le jour 3 à partir des montants 20 et 0.
3. Comparez le résultat aux moyennes précédentes sur trois jours.

Corrigé expliqué : une ligne précédente signifie deux jours

Le nouveau cadre commence à 1 PRECEDING. Ses six moyennes sont 10.00, 15.00, 10.00, 20.00, 45.00 et 25.00. Le jour 5 utilise 40 et 50 ; le jour 6 utilise 50 et 0. Les jours sans vente restent dans le dénominateur, car ce sont des jours calendaires confirmés dans ce rapport, et non des observations absentes à ignorer.

SQL / lire l’instruction une clause à la fois

```sql
WITH daily(d,amount) AS (
 VALUES (1,10),(2,20),(3,0),
        (4,40),(5,50),(6,0)
)
SELECT d,ROUND(AVG(amount) OVER (
 ORDER BY d ROWS BETWEEN 1 PRECEDING
 AND CURRENT ROW),2) AS moving_avg
FROM daily
ORDER BY d;
```

Résultat / données d’exemple

d	moving_avg
1	10.00
2	15.00
3	10.00
4	20.00
5	45.00
6	25.00

Une CTE récursive étend à répétition son résultat précédent

Comment une requête peut-elle parcourir un nombre inconnu de niveaux hiérarchiques ? Une CTE récursive part d’une ancre, puis applique à répétition une étape aux lignes nouvellement trouvées. WITH RECURSIVE permet à la CTE de se référencer elle-même. UNION ALL ajoute les lignes de chaque tour sans éliminer les doublons. Cet exemple de comptage part de 1, ajoute 1 tant que n est inférieur à 5 et s’arrête quand l’étape ne renvoie plus de ligne.

SQL / lire l’instruction une clause à la fois

```sql
WITH RECURSIVE numbers(n) AS (
 VALUES (1)
 UNION ALL
 SELECT n+1 FROM numbers
 WHERE n<5
)
SELECT n FROM numbers
ORDER BY n;
```

Sens des mots-clés

RECURSIVE — Permet à une requête WITH de lire son propre tour précédent.

ancre — Lignes initiales, écrites avant UNION ALL.

étape récursive — Requête produisant le prochain tour de lignes.

UNION ALL — Ajoute les lignes en conservant les doublons.

terminaison — Moment où l’étape ne renvoie plus de nouvelles lignes.

Suivre un tour récursif à la fois

L’ancre fournit 1. L’étape produit ensuite 2, 3, 4 et 5 lors de tours successifs. Lorsque la ligne de travail courante vaut 5, n < 5 est faux : l’étape ne produit donc rien et la récursivité s’achève. ORDER BY contrôle la séquence affichée. La condition d’arrêt est essentielle ; espérer simplement que le nombre de lignes souhaité a été produit ne sécuriserait pas une étape non bornée.

Résultat / données d’exemple

n
1
2
3
4
5

Suivre un tour récursif à la fois / raisonnement pas à pas

Suivez ces étapes en parallèle de l’exemple SQL précédent. Prévoyez ce que chaque étape reçoit et produit avant de l’exécuter. L’essentiel n’est pas seulement de savoir si l’instruction s’exécute, mais pourquoi ces opérations répondent à la question posée avec les données fournies.

Suivez le raisonnement

1. L’ancre fournit la première ligne de travail.
2. Chaque étape réussie ajoute une ligne.
3. Une étape vide termine la récursivité.

Parcourir les enfants des catégories CampusMarket

La table des catégories ne stocke que le parent immédiat de chaque catégorie. L’ancre sélectionne la racine, dont parent_id vaut NULL, et lui attribue une profondeur de 0. Chaque étape récursive joint les parents nouvellement trouvés à leurs enfants et augmente la profondeur de un. Le résultat final inclut les descendants directs et indirects. Ce jeu de données de référence n’a pas de cycle : le processus s’arrête donc lorsque Accessories et Stationery n’ont plus d’enfants.

SQL / lire l’instruction une clause à la fois

```sql
WITH RECURSIVE tree(category_id,name,depth) AS (
 SELECT category_id,name,0
 FROM campus.categories WHERE parent_id IS NULL
 UNION ALL
 SELECT c.category_id,c.name,t.depth+1
 FROM campus.categories AS c
 JOIN tree AS t ON c.parent_id=t.category_id
)
SELECT category_id,name,depth FROM tree
ORDER BY depth,category_id;
```

La profondeur compte les étapes de parent à enfant

All est la racine à la profondeur 0. Technology et Lifestyle se trouvent une étape plus bas ; Accessories et Stationery, deux étapes plus bas. La requête découvre les cinq catégories sans coder en dur un nombre maximal de jointures. L’ordre affiché est explicitement en largeur par profondeur, puis par identifiant ; il ne dépend pas d’une promesse non documentée sur l’ordre naturel de sortie du moteur récursif.

Résultat / données d’exemple

category_id	name	depth
1	All	0
2	Technology	1
3	Lifestyle	1
4	Accessories	2
5	Stationery	2

Détecter un nœud répété avant de le développer de nouveau

Un cycle survient lorsque le suivi des liens ramène à un nœud déjà présent sur le même chemin. La clause CYCLE de PostgreSQL le suit pour nous. CYCLE id identifie la valeur à vérifier, SET is_cycle nomme un indicateur booléen et USING path nomme la séquence des nœuds visités. La requête ci-dessous contient les liens de 2 vers 3 et de 3 vers 2 : elle doit donc cesser de développer cet itinéraire répétitif.

SQL / lire l’instruction une clause à la fois

```
WITH RECURSIVE edges(a,b) AS (
 VALUES (1,2),(2,3),(3,2),(1,4)
), walk(id) AS (
 VALUES (1)
 UNION ALL
 SELECT e.b FROM walk AS w
 JOIN edges AS e ON e.a=w.id
) CYCLE id SET is_cycle USING path
SELECT id,is_cycle,path FROM walk
ORDER BY path;
```

Sens des mots-clés

cycle — Itinéraire qui revisite un nœud déjà présent sur cet itinéraire.

CYCLE id — Vérifie si id se répète sur l’itinéraire courant.

SET is_cycle — Nomme l’indicateur de cycle vrai ou faux généré.

USING path — Nomme la séquence générée des nœuds visités.

tableau — Collection ordonnée de valeurs, utilisée ici pour enregistrer un itinéraire.

L’itinéraire répétitif est visible et fini

L’itinéraire de 1 à 2 à 3 atteint à nouveau 2 : sa ligne de fermeture est donc marquée true. Cette ligne n’est pas développée en un autre 3. L’itinéraire indépendant de 1 à 4 reste valide. La colonne path montre l’itinéraire ayant produit chaque ligne ; PostgreSQL affiche ces chemins comme des tableaux d’enregistrements à un seul champ. Un WHERE NOT is_cycle final pourrait masquer les lignes fermant les cycles après leur rôle de diagnostic.

Résultat / données d’exemple

id	is_cycle	path
1	FALSE	{(1)}
2	FALSE	{(1),(2)}
3	FALSE	{(1),(2),(3)}
2	TRUE	{(1),(2),(3),(2)}
4	FALSE	{(1),(4)}

La terminaison doit découler des données et des règles de la requête, avec gestion des cycles si nécessaire.

LATERAL exécute une requête pour chaque ligne source

Quelle est la dernière commande payée de chaque étudiant ? Une sous-requête LATERAL dans FROM peut lire les colonnes d’un élément FROM précédent. La requête interne peut ainsi utiliser l’identifiant de l’étudiant courant, trier ses commandes et n’en garder qu’une. Un LEFT JOIN LATERAL conserve aussi les étudiants pour lesquels la requête interne ne trouve aucune commande. C’est utile quand chaque ligne source exige son propre résultat limité.

LATERAL permet à une sous-requête FROM d’utiliser les valeurs d’un élément FROM précédent.

Sens des mots-clés

LATERAL — Permet à cette requête FROM de référencer une ligne source précédente.

LEFT JOIN LATERAL — Conserve la ligne source même si sa requête par ligne ne renvoie rien.

ON true — Accepte comme correspondance chaque ligne produite par la requête latérale.

Trouver une dernière commande payée par étudiant

Pour chaque étudiant s, le WHERE interne limite les commandes à cet étudiant et au statut paid. ordered_at décroissant place l'achat le plus récent en premier ; order_id décroissant départage de façon cohérente les ex æquo. LIMIT 1 renvoie au plus une commande. ON true n'ajoute aucune condition de correspondance, car la sous-requête assure déjà celle de l'étudiant. Le LEFT JOIN fournit NULL lorsque la sous-requête ne renvoie aucune ligne.

SQL / lire l'instruction une clause à la fois

```sql
SELECT s.student_id,s.full_name,last_order.order_id
FROM campus.students AS s
LEFT JOIN LATERAL (
 SELECT o.order_id
 FROM campus.orders AS o
 WHERE o.student_id=s.student_id
  AND o.status='paid'
 ORDER BY o.ordered_at DESC,o.order_id DESC
 LIMIT 1
) AS last_order ON true
ORDER BY s.student_id;
```

Farah reste même sans commande correspondante

La dernière commande payée d’Alice est 103, celle de Bilal 107 et celle de Chloe 108. La commande pending de Diego est ignorée, laissant la commande payée 110. La commande cancelled d’Eva est également ignorée, laissant 109. Farah reste avec NULL, car la jointure gauche conserve sa ligne d’étudiante. Le résultat contient exactement une ligne par étudiant, car la sous-requête latérale renvoie au plus une commande correspondante.

Résultat / données d’exemple

student_id	full_name	order_id
1	Alice Martin	103
2	Bilal Amrani	107
3	Chloe Bernard	108
4	Diego Costa	110
5	Eva Dupont	109
6	Farah Idrissi	NULL

Un rapport peut contenir plusieurs niveaux de synthèse

Un responsable veut les totaux par campus et un total général dans le même rapport. GROUPING SETS énumère les regroupements distincts à calculer sur la même entrée. Des parenthèses autour de campus demandent des groupes par campus ; des parenthèses vides demandent un total général. Nos quatre lignes de ventes simplifiées couvrent deux campus et deux catégories, ce qui permet de vérifier manuellement chaque sous-total avant d’aborder un rapport métier plus vaste.

GROUPING SETS demande plusieurs synthèses GROUP BY distinctes dans un seul résultat.

Résultat / données d’exemple

campus	category	amount
north	Books	10
north	Tech	40
south	Books	30
south	Tech	20

Un rapport peut contenir plusieurs niveaux de synthèse / guide des mots-clés

Utilisez ce vocabulaire pour lire l’exemple précédent. Chaque définition explique le rôle exact d’un mot ou symbole dans l’instruction. Relisez le SQL depuis sa table d’entrée jusqu’à ses conditions et sa sortie, en utilisant ces définitions pour expliquer ce qui est demandé au serveur.

Sens des mots-clés

ensemble de regroupement — Ensemble choisi de colonnes servant à former des groupes de synthèse.

() — Ensemble de regroupement vide, produisant un total général.

sous-total — Total pour une partie d’un regroupement plus large.

GROUPING marque les colonnes omises par un niveau de synthèse

Cette requête demande les totaux par campus et le total général. GROUPING(campus) renvoie 0 lorsque campus participe au regroupement de cette ligne et 1 s’il en est omis. Une colonne de regroupement omise apparaît comme NULL : c’est un marqueur de synthèse, pas nécessairement une donnée source manquante. L’indicateur permet au rapport de distinguer ces sens sans les deviner à partir de la seule valeur affichée.

SQL / lire l’instruction une clause à la fois

```sql
WITH sales(campus,category,amount) AS (
 VALUES ('north','Books',10),('north','Tech',40),
        ('south','Books',30),('south','Tech',20)
)
SELECT campus,SUM(amount) AS total,
 GROUPING(campus) AS all_campuses
FROM sales
GROUP BY GROUPING SETS ((campus),())
ORDER BY all_campuses,campus;
```

Sens des mots-clés

GROUPING SETS — Énumère les différents ensembles de colonnes de regroupement à calculer.

GROUPING(column) — Renvoie 1 si ce regroupement omet la colonne, sinon 0.

La ligne du total général a une granularité différente

north et south totalisent chacun 50. La ligne où all_campuses vaut 1 totalise les quatre ventes d’entrée, soit 100. Sa cellule campus vaut NULL, car ce regroupement n’a pas de clé campus. Additionner les trois totaux affichés donnerait 200 et compterait deux fois les mêmes ventes. Les rapports mêlant les niveaux exigent des libellés de niveau clairs pour éviter de traiter sous-totaux et totaux généraux comme des faits indépendants.

Résultat / données d’exemple

campus	total	all_campuses
north	50	0
south	50	0
NULL	100	1

Utilisez les indicateurs de regroupement pour expliquer les NULL de synthèse et les sens mêlés des lignes.

ROLLUP suit les préfixes ; CUBE inclut chaque sous-ensemble

ROLLUP(campus,category) demande les détails par campus et catégorie, les sous-totaux par campus et un total général. Il suit la hiérarchie indiquée : inverser les colonnes change donc le sens des sous-totaux. CUBE(campus,category) demande aussi les totaux par catégorie seule, car il inclut chaque sous-ensemble des deux colonnes. La requête ci-dessous utilise CUBE et des indicateurs GROUPING séparés pour lire explicitement le niveau de chaque ligne du résultat.

SQL / lire l’instruction une clause à la fois

```sql
WITH sales(campus,category,amount) AS (
 VALUES ('north','Books',10),('north','Tech',40),
        ('south','Books',30),('south','Tech',20)
)
SELECT campus,category,SUM(amount) AS total,
 GROUPING(campus) AS gc,GROUPING(category) AS gk
FROM sales
GROUP BY CUBE(campus,category)
ORDER BY gc,gk,campus,category;
```

Sens des mots-clés

ROLLUP(a,b) — Demande (a,b), puis (a), puis le total général.

CUBE(a,b) — Demande (a,b), (a), (b) et le total général.

Seul CUBE ajoute les totaux par catégorie tous campus confondus

Les quatre lignes de détail ont leurs deux indicateurs à 0. Les sous-totaux par campus ont gc = 0 et gk = 1. Les totaux par catégorie seule ont gc = 1 et gk = 0 : Books totalise 40 et Tech 60 sur les deux campus. La dernière ligne a ses deux indicateurs à 1. ROLLUP(campus,category) renverrait les mêmes lignes, sauf les deux totaux par catégorie seule.

Résultat / données d’exemple

campus	category	total	gc	gk
north	Books	10	0	0
north	Tech	40	0	0
south	Books	30	0	0
south	Tech	20	0	0
north	NULL	50	0	1
south	NULL	50	0	1
NULL	Books	40	1	0

Seul CUBE ajoute les totaux par catégorie tous campus confondus / résultat commenté

Les quatre lignes de détail ont leurs deux indicateurs à 0. Les sous-totaux par campus ont gc = 0 et gk = 1. Les totaux par catégorie seule ont gc = 1 et gk = 0 : Books totalise 40 et Tech 60 sur les deux campus. La dernière ligne a ses deux indicateurs à 1. ROLLUP(campus,category) renverrait les mêmes lignes, sauf les deux totaux par catégorie seule.

Résultat / données d’exemple

campus	category	total	gc	gk
NULL	Tech	60	1	0
NULL	NULL	100	1	1

À vous de jouer : demander les totaux par catégorie et le total général

Utilisez les quatre mêmes lignes de vente. Le responsable des catégories veut les totaux Books et Tech ainsi qu’un total général, sans ligne au niveau campus. Écrivez une liste GROUPING SETS explicite plutôt que CUBE et incluez GROUPING(category) avec l’alias all_categories. Prévoyez les trois montants et leurs indicateurs. Expliquez pourquoi campus doit être absent du SELECT lorsque les niveaux demandés ne regroupent jamais par campus.

Suivez le raisonnement

1. Sélectionnez category, SUM(amount) et GROUPING(category).
2. Demandez le regroupement par catégorie et le regroupement vide.
3. Placez les lignes de catégorie avant la ligne du total général.

Corrigé expliqué : ne lister que les niveaux requis

GROUPING SETS ((category),()) renvoie Books à 40, Tech à 60 et le total général à 100. Les lignes de catégorie ont all_categories = 0 et le total général a 1. Les ensembles de regroupement explicites sont utiles lorsqu'un rapport exige un petit choix de niveaux. La sortie doit rendre visible le sens de chaque sous-total avant l'export ou la création d'un graphique.

SQL / lire l'instruction une clause à la fois

```sql
WITH sales(campus,category,amount) AS (
 VALUES ('north','Books',10),('north','Tech',40),
        ('south','Books',30),('south','Tech',20)
)
SELECT category,SUM(amount) AS total,
 GROUPING(category) AS all_categories
FROM sales
GROUP BY GROUPING SETS ((category),())
ORDER BY all_categories,category;
```

Résultat / données d'exemple

category	total	all_categories
Books	40	0
Tech	60	0
NULL	100	1

Approfondissement guidé : définir une cohorte avant de la mesurer

Une cohorte est un groupe défini par un événement initial commun. Pour la fidélisation des achats, affectez chaque personne au mois de son premier achat payé, puis demandez combien reviennent les mois suivants. La fidélisation signifie une activité poursuivie selon une règle précisée. Nos six lignes simplifiées sont des achats aux indices de mois 0, 1 et 2 ; ces indices rendent la soustraction lisible avant d’utiliser de vraies dates calendaires.

Résultat / données d’exemple

person	month_index
1	0
1	1
2	0
3	1
3	2
4	1

Approfondissement guidé : définir une cohorte avant de la mesurer / guide des mots-clés

Utilisez ce vocabulaire pour lire l’exemple précédent. Chaque définition explique le rôle exact d’un mot ou symbole dans l’instruction. Relisez le SQL depuis sa table d’entrée jusqu’à ses conditions et sa sortie, en utilisant ces définitions pour expliquer ce qui est demandé au serveur.

Sens des mots-clés

cohorte — Personnes partageant un événement ou une période de départ définis.

fidélisation — Poursuite d’activité mesurée selon une règle explicite.

indice de mois — Entier identifiant un mois dans l’ordre chronologique.

ancienneté — Mois écoulés depuis le mois de départ de cette personne.

Attribuer le premier mois, puis compter les personnes par ancienneté de cohorte

La CTE firsts trouve le premier mois d’achat de chaque personne avec MIN. Sa jointure avec purchases rattache ce mois de départ à chaque achat. Soustraire first_mo de m donne l’ancienneté dans la cohorte : zéro est le mois de départ, un le mois suivant. COUNT(DISTINCT person) compte les personnes actives plutôt que les achats : plusieurs achats d’une personne dans un mois ne compteraient donc qu’une fois.

SQL / lire l’instruction une clause à la fois

```sql
WITH buys(person,m) AS (
 VALUES (1,0),(1,1),(2,0),
        (3,1),(3,2),(4,1)
), firsts AS (
 SELECT person,MIN(m) AS first_mo
 FROM buys GROUP BY person
)
SELECT f.first_mo AS cohort,m-f.first_mo AS age,
 COUNT(DISTINCT b.person) AS active_people
FROM buys AS b JOIN firsts AS f USING (person)
GROUP BY f.first_mo,m-f.first_mo
ORDER BY cohort,age;
```

Les deux cohortes ont une personne qui revient

Chaque cohorte commence avec deux personnes à l’ancienneté 0 et compte une personne active à l’ancienneté 1. Diviser l’effectif d’ancienneté 1 par celui d’ancienneté 0 de sa cohorte donne 50% de retour à cette ancienneté. La requête ne renvoie que l’activité observée ; l’absence d’une ligne cohorte-ancienneté ne prouve pas automatiquement une fidélisation nulle. Une grille de fidélisation complète exige aussi des périodes d’observation valides et un recul suffisant pour chaque cohorte.

Résultat / données d’exemple

cohort	age	active_people
0	0	2
0	1	1
1	0	2
1	1	1

Définissez l’événement initial, l’événement actif, le dénominateur et la période d’observation avant de qualifier un chiffre de fidélisation.

Approfondissement guidé : trouver les séries de jours consécutifs

Un îlot est une suite d’activité consécutive ; une lacune sépare une suite de la suivante. Supposons qu’une personne ait été active aux jours numérotés 1, 2, 3, 6, 7 et 10. Dans une suite consécutive, le numéro du jour et le numéro de ligne augmentent tous deux de un : leur différence reste donc constante. Cette constante peut identifier la suite, qu’un GROUP BY ultérieur résume par son début, sa fin et sa longueur.

Résultat / données d’exemple

day_number
1
2
3
6
7
10

Approfondissement guidé : trouver les séries de jours consécutifs / guide des mots-clés

Utilisez ce vocabulaire pour lire l’exemple précédent. Chaque définition explique le rôle exact d’un mot ou symbole dans l’instruction. Relisez le SQL depuis sa table d’entrée jusqu’à ses conditions et sa sortie, en utilisant ces définitions pour expliquer ce qui est demandé au serveur.

Sens des mots-clés

îlot — Suite de valeurs consécutives selon une règle précisée.

lacune — Interruption entre des suites consécutives.

série — Suite de jours d’activité consécutifs.

clé d’îlot — Valeur calculée commune à toutes les lignes d’une suite consécutive.

Étiqueter chaque suite avant de la résumer

ROW_NUMBER ordonne les six numéros de jours actifs distincts. Le soustraire de d produit les clés 0, 0, 0, 2, 2 et 4. La CTE labeled expose ces clés au GROUP BY externe. MIN et MAX trouvent les limites de la suite et COUNT donne son nombre de jours. Ce modèle repose sur des entiers consécutifs par pas de un ; une autre définition de l’activité consécutive exige une autre règle.

SQL / lire l’instruction une clause à la fois

```sql
WITH active(d) AS (
 VALUES (1),(2),(3),(6),(7),(10)
), labeled AS (
 SELECT d,d-ROW_NUMBER() OVER (ORDER BY d) AS k
 FROM active
)
SELECT MIN(d) AS first_day,MAX(d) AS last_day,
 COUNT(*) AS days
FROM labeled
GROUP BY k
ORDER BY first_day;
```

Trois îlots expliquent la répartition de l’activité

La personne a une série de trois jours du jour 1 au jour 3, une série de deux jours du jour 6 au jour 7 et une série d’un seul jour au jour 10. Les plages de jours manquants créent les limites. Ce résultat résume l’activité consécutive plutôt que la seule activité totale : un total de six jours actifs masquerait la différence entre une longue série et plusieurs visites séparées.

Résultat / données d’exemple

first_day	last_day	days
1	3	3
6	7	2
10	10	1

Explicitez la règle des unités consécutives et la granularité d’entrée avant d’utiliser un modèle de séries.

Choisir les étapes selon la question, puis vérifier de petits résultats

Les outils du jour répondent à différents besoins : les CTE nomment les étapes, les fenêtres conservent le détail en comparant les lignes, les CTE récursives suivent les relations répétées, LATERAL fournit une requête par ligne source et les ensembles de regroupement produisent des niveaux de synthèse choisis. En TP, partez d’une question métier et précisez ce que représente une ligne de résultat. Prévoyez un petit résultat exact avant d’étendre la requête au jeu de données de référence CampusMarket.

Suivez le raisonnement

1. Calculez le chiffre d’affaires des commandes payées à partir des prix historiques des lignes et vérifiez le total de 475.
2. Précisez la partition, le tri et le cadre avant d’écrire un calcul de fenêtrage.
3. Vérifiez la terminaison de la récursivité et la conservation des lignes sources sans correspondance.
4. Étiquetez chaque niveau de synthèse et définissez les prérequis des approfondissements avant de combiner les modèles.

Des étapes lisibles et de petites sorties exactes rendent les requêtes avancées testables et explicables.